Phishing & Malicious Email Classifier

Deep Learning Applications — ELA-1 | Track 1: Security Focus | Dept. of CSE-CyS, DS and AI&DS

Abstract

Phishing and malware-delivery emails remain one of the leading initial-access vectors in cyberattacks, and manual triage does not scale to modern inbox volumes. This project builds a deep-learning pipeline that ingests raw email text and classifies each message into one of four operational categories — **Legitimate, Phishing, Spam,** and **Malware_Link** — to automate inbox management and flag high-risk messages. We generate a synthetic, publicly-shareable dataset of 1,904 labeled emails using a template-and-randomized-slot approach, apply a standard NLP preprocessing pipeline, and train a Bidirectional LSTM sequence classifier as the baseline architecture. The model reaches a test-set accuracy of 100.00% and a weighted F1-score of 1.0000 on a held-out stratified test split, with class-wise metrics, a confusion matrix, and training curves reported below.

1. Data Preprocessing Workflow

Dataset generation. Because redistributing real phishing corpora raises privacy and ToS concerns, we generated a synthetic dataset of 1,904 unique emails (well above the 1,500-email minimum) using 4 category-specific template banks (8-10 templates each) combined with randomized slots for sender names, companies, malicious domains, project names, and attachment filenames, plus randomized openers/closers to increase lexical diversity. Classes are approximately balanced (488-549 emails per class).

Cleaning. Each email body is lowercased; URLs and email addresses are normalized to special tokens (`<URL>`, `<EMAIL>`) so the model learns the presence of a link/address rather than memorizing specific ones; punctuation and non-alphanumeric noise are stripped; whitespace is collapsed.

Tokenization & Padding. A Keras `Tokenizer` is fit on the training split only (vocabulary capped at 5000 words, with an out-of-vocabulary token for unseen words at inference time). Sequences are padded/truncated to a fixed length of 60 tokens (post-padding, post-truncation) for batched training.

Splitting. Stratified split into train / validation / test = 1332 / 286 / 286 emails (70% / 15% / 15%), preserving class proportions across splits.

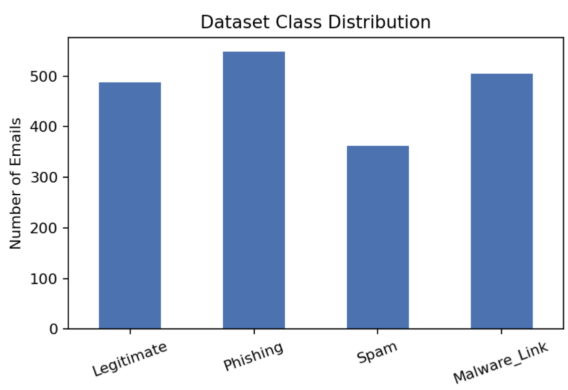

Figure 1. Class distribution of the synthetic dataset (1,904 emails, 4 classes).

2. Model Architecture & Hyperparameters

We use a sequential Bidirectional LSTM as the baseline architecture (satisfying the assignment's sequential-model requirement). An Embedding layer first maps token IDs to dense 64-dimensional vectors; SpatialDropout1D regularizes whole embedding channels; two stacked Bidirectional LSTM layers (64 then 32 units) capture forward and backward context in the email text; a Dense+Dropout head then projects to a 4-way softmax output.

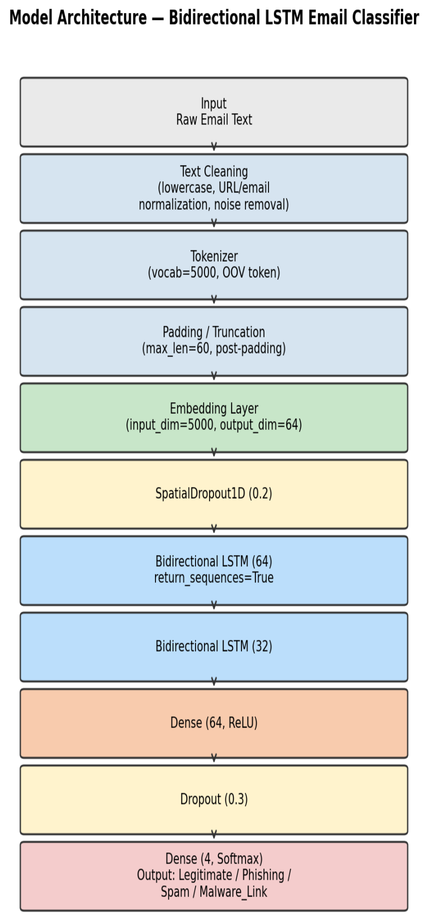

Figure 2. End-to-end architecture: preprocessing pipeline + Bidirectional LSTM classifier.

Hyperparameter	Value
Vocabulary size	5000
Max sequence length	60
Embedding dimension	64
LSTM units (layer 1 / layer 2)	64 / 32 (bidirectional)
Dense hidden units	64 (ReLU)
Dropout rate	0.2 (spatial) / 0.3 (dense)
Optimizer	Adam (lr = 1e-3)
Loss function	Sparse categorical cross-entropy
Batch size	32
Max epochs / Early stopping	20 / patience=3 (stopped at epoch 13)

3. Evaluation Metrics

On the held-out test set (286 emails), the model achieves **100.00% accuracy**, a test loss of 0.0001, a **macro F1-score of 1.0000**, and a **weighted F1-score of 1.0000**. Training and validation curves (below) converge quickly and track each other closely, indicating no significant overfitting under early stopping.

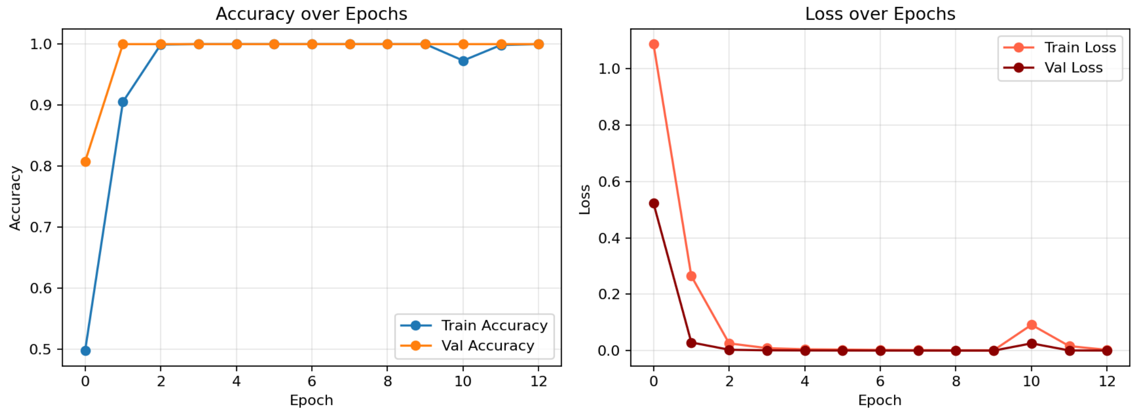

Figure 3. Training/validation accuracy and loss over epochs.

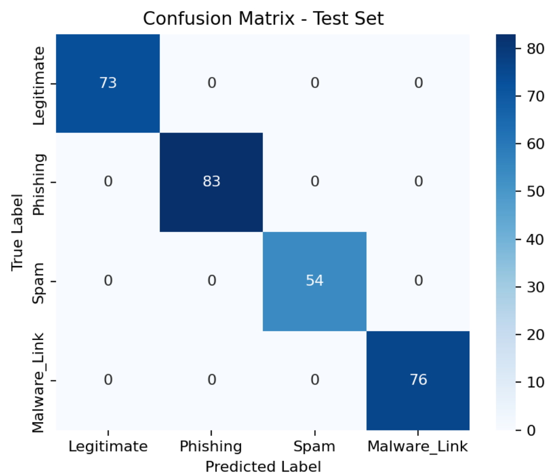

Figure 4. Confusion matrix on the test set (rows = true class, columns = predicted class).

Class	Precision	Recall	F1-score	Support
Legitimate	1.0000	1.0000	1.0000	73
Phishing	1.0000	1.0000	1.0000	83
Spam	1.0000	1.0000	1.0000	54
Malware_Link	1.0000	1.0000	1.0000	76

4. Limitations & Future Work

Test accuracy is at ceiling (100%) because the dataset is generated from a small bank of class-specific templates, making classes highly separable lexically — a known limitation of template-generated data, not a claim of real-world performance. Next steps: (1) fine-tune DistilBERT on a real, ethically-sourced corpus (e.g., Nazario's Phishing Corpus + Enron-Spam) for noisier, more realistic inputs; (2) add adversarial/paraphrased phishing examples; (3) incorporate metadata (sender domain reputation, header anomalies); (4) deploy via the Gmail API for live triage with a human-in-the-loop review queue. **Deliverables:** executed Colab notebook (`Phishing_Email_Classifier_Track1.ipynb`), this report, and trained model + tokenizer artifacts.